

Multi-Agent 기반 퇴사자 데이터 유출 탐지 시스템 — 에이전트 설계 문서

이 문서는 시스템을 처음 보는 개발자가 각 에이전트를 독립적으로 구현할 수 있도록 Input / Output / 판단 기준 / DB 쿼리 / 프롬프트 방향을 모두 명시합니다.

목차

1. 시스템 개요
2. Deep Agent란 무엇인가 — Supervisor 패턴
3. 전체 데이터 흐름
4. 공통 데이터 구조 정의
5. MAIN AGENT
6. STEP 1 — Baseline Agent
7. STEP 2 — 유출 행위 분석 Agent
8. STEP 3 — 민감 파일 분류 Agent
9. STEP 4 — 행동 패턴 분석 Agent
10. STEP 5 — Counter-evidence Agent
11. 리스크 스코어링 (Main Agent 내부)
12. 최종 출력 포맷
13. 구현 기술 스택
14. Tool 함수 명세

1. 시스템 개요

목적

퇴사자의 C드라이브 이미지를 분석해 **데이터 유출 여부를 자동으로 판정**하고, 유출이 있다면 **어떤 파일이, 어떤 경로로, 언제 나갔는지** 하이라이트해서 보고서를 생성한다.

입력

- 퇴사자 기본 정보: 이름, 직급, 입사일, 퇴사일
- C드라이브 이미지 파일

출력

- 유출 있음:** 의심 파일 하이라이트 + 유출 경로 시각화 + Evidence Network
- 유출 없음:** 클린 인증서

데이터베이스 구성

DB	용도	주요 테이블
PostgreSQL	정형 데이터	files, email_messages, activity_events, entity_canonical, file_access_logs, messenger_logs, deleted_files
Qdrant	벡터 검색	hyena_content_chunks (26,485개 벡터)
Neo4j	관계 그래프	GNode (41,959개 노드, 49,430개 관계)

2. Deep Agent란 무엇인가

핵심 개념

Deep Agent는 **LangChain/LangGraph 기반의 에이전트 아키텍처**로, 단순히 Tool을 반복 호출하는 "Shallow Agent"의 한계를 넘기 위해 설계된 방식이다.

[Shallow Agent – 단순 Tool 반복 호출]

LLM → Tool 호출 → 결과 → Tool 호출 → 결과 → ...

→ 복잡한 작업에서 계획 없이 진행해 중간 단계를 빠트리거나 실패함

[Deep Agent – 계획 → 실행 → 반환]

LLM이 먼저 "무엇을 어떤 순서로 할지" 계획을 수립하고

순서대로 Tool을 실행한 뒤 결과를 구조화해서 반환함

→ 복잡한 다단계 작업도 안정적으로 처리

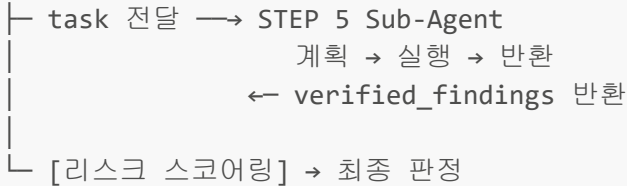
우리 시스템의 Deep Agent 구조 — Supervisor 패턴

Main Agent = Supervisor (지휘관) STEP 1~5 = Sub-Agent (전문가)

Main Agent가 각 Sub-Agent에게 명시적으로 task를 전달하고, Sub-Agent는 받은 task를 계획→실행→반환으로 처리한 뒤 결과를 Main Agent에게 돌려준다. Main Agent는 모든 결과를 수집해서 교차 대조 → 스코어링 → 최종 판정을 수행한다.

Main Agent (Supervisor)

- task 전달 → STEP 1 Sub-Agent
 - 계획 → 실행 → 반환
 - ← baseline_profile 반환
- task 전달 → STEP 2 Sub-Agent (병렬)
- task 전달 → STEP 3 Sub-Agent (병렬)
- task 전달 → STEP 4 Sub-Agent (병렬)
 - 각자 계획 → 실행 → 반환
 - ← 결과 3개 반환
- [교차 대조] STEP 2 + STEP 3 결과 직접 처리



단, **Sub-Agent**를 어떤 순서로 실행할지는 코드(**LangGraph**)에 고정된다. Main Agent가 "다음에 뭘 해야 할지" LLM으로 결정하지 않는다. 이 구조 덕분에 포렌식 분석의 재현성과 신뢰성이 보장된다.

Sub-Agent(STEP 1~5)는 각각 독립적인 **Deep Agent**다. Main으로부터 task를 받아 내부적으로 계획→실행→반환으로 동작한다.

규칙 기반 코드 vs Deep Agent 비교

[규칙 기반]
if sender contains "protonmail":
 flag = True
→ 개발자가 패턴을 직접 코딩. 새로운 수법은 코드 수정 필요.

[Deep Agent]
System Prompt: "유출 의심 채널을 탐지하라. 익명 채널, 개인 Gmail, 첨부파일 외부 발송 등을 종합적으로 판단해라."
LLM이 → 계획 수립 → Tool 호출 → 맥락 기반 판단 → 결과 반환
→ 새로운 유출 패턴도 프롬프트 수정만으로 대응 가능.
→ "설에 한번 뵈죠^^" 같은 맥락 의존적 표현도 판단 가능.

Sub-Agent 내부 동작 흐름 예시 (STEP 2 기준)

Main Agent → STEP 2에 task 전달
 "이지수 퇴사 전 90일, 기준선 참고해서 유출 채널 탐지해줘"

↓

[계획] STEP 2 Sub-Agent가 수신 후 분석 순서 결정
"1) 익명 채널 이메일 먼저 (위험도 높음)
2) 기준선 대비 외부 발신량 비교
3) 메신저 민감 키워드 탐지"

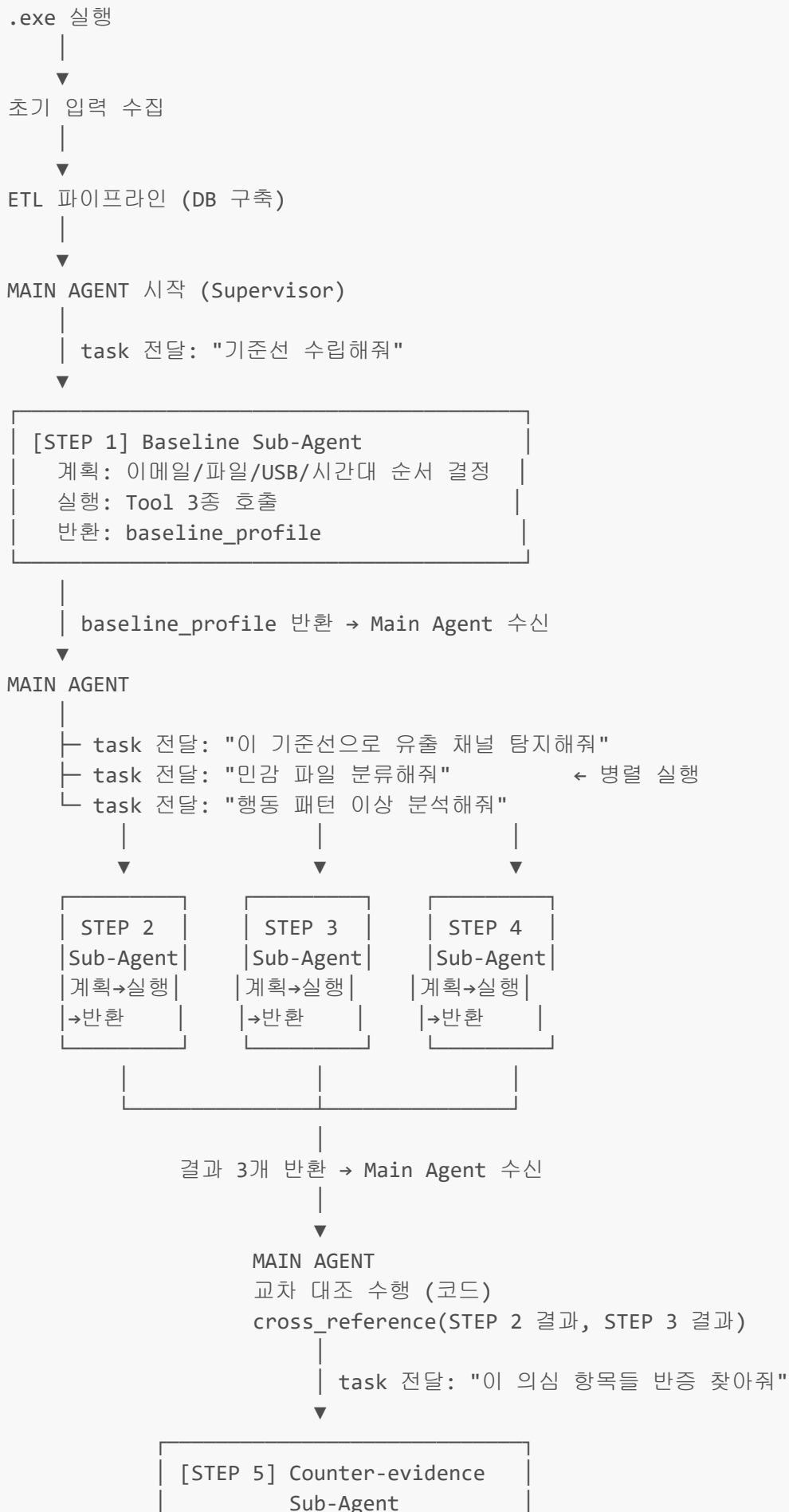
↓

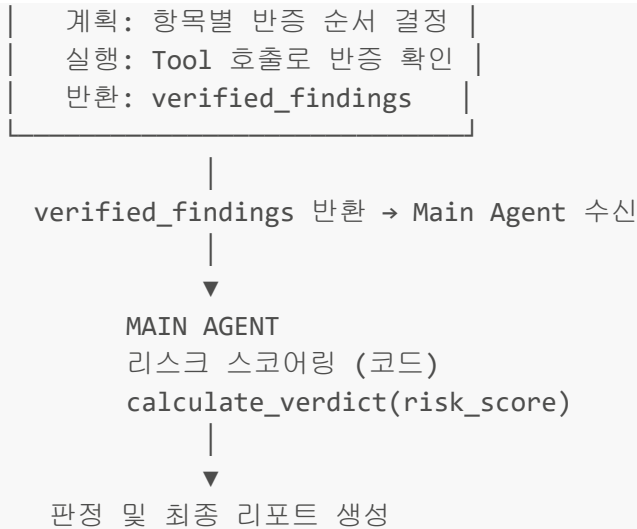
[실행]
→ get_anonymous_channel_emails() 호출 → DB 결과 수신
→ get_external_emails() 호출 → 기준선과 비교
→ get_messenger_logs() 호출 → 키워드 탐지

↓

[반환] suspicious_channels 리스트 구조화
→ Main Agent에게 결과 반환
→ Main Agent가 State에 저장 후 다음 단계 진행

3. 전체 데이터 흐름





4. 공통 데이터 구조 정의

모든 에이전트가 공유하는 **State** 객체:

```

class InvestigationState(TypedDict):
    # 초기 입력
    subject_name: str          # 퇴사자 이름 (예: "이지수")
    subject_position: str      # 직급 (예: "과장")
    hire_date: str             # 입사일 (예: "2019-03-01")
    resignation_date: str      # 퇴사일 (예: "2021-02-28")
    analysis_start: str        # 분석 시작일 = resignation_date - 90일
    source_label: str          # 데이터 소스 레이블 (예: "HYENA CTF")

    # 각 에이전트 출력 (순서대로 채워짐)
    baseline_profile: dict     # STEP 1 결과
    suspicious_channels: list  # STEP 2 결과
    sensitive_files: list      # STEP 3 결과
    behavior_anomalies: dict   # STEP 4 결과
    cross_reference: list      # Main Agent 교차 대조 결과
    verified_findings: list    # STEP 5 결과
    risk_score: int            # 리스크 점수
    verdict: str               # HIGH / MEDIUM / LOW / CLEAN
    final_report: dict         # 최종 리포트
  
```

5. MAIN AGENT

역할

Supervisor로서 전체 수사 흐름을 지휘한다. 각 Sub-Agent에게 명시적으로 task를 전달하고, 반환된 결과를 수집해서 교차 대조와 리스크 스코어링을 직접 수행한 뒤 최종 판정을 내린다.

Supervisor 동작 순서

1. STEP 1 Sub-Agent에게 task 전달 → baseline_profile 수신
2. STEP 2 / STEP 3 / STEP 4 Sub-Agent에게 동시에 task 전달 (병렬)
→ suspicious_channels / sensitive_files / behavior_anomalies 수신
3. cross_reference(STEP 2 결과, STEP 3 결과) 직접 수행 (코드)
4. STEP 5 Sub-Agent에게 task 전달 (교차 대조 결과 포함) → verified_findings 수신
5. calculate_verdict(risk_score) 직접 수행 (코드)
6. 최종 리포트 생성 (LLM 1회 호출)

Sub-Agent에게 전달하는 task 형식

```
# STEP 1 task
{
  "task": "baseline_profile 수립",
  "subject_name": state["subject_name"],
  "analysis_start": state["analysis_start"],
  "resignation_date": state["resignation_date"],
  "source_label": state["source_label"]
}

# STEP 2 task
{
  "task": "유출 채널 탐지",
  "subject_name": state["subject_name"],
  "baseline_profile": state["baseline_profile"], # STEP 1 결과 포함
  "analysis_start": state["analysis_start"],
  "resignation_date": state["resignation_date"]
}

# STEP 5 task
{
  "task": "의심 항목 반증 검증",
  "subject_name": state["subject_name"],
  "suspicious_channels": state["suspicious_channels"],
  "sensitive_files": state["sensitive_files"],
  "behavior_anomalies": state["behavior_anomalies"],
  "cross_reference": state["cross_reference"] # 교차 대조 결과 포함
}
```

System Prompt (최종 리포트 생성 시)

당신은 디지털 포렌식 전문가입니다.
퇴사자의 데이터 유출 여부를 분석하는 수사 시스템을 운영합니다.

분석 대상: {subject_name} ({subject_position})
입사일: {hire_date} / 퇴사일: {resignation_date}
분석 기간: {analysis_start} ~ {resignation_date} (최근 90일)

각 Sub-Agent로부터 수집된 결과를 바탕으로 최종 수사 보고서를 작성하세요.

판단 원칙:

- 단순 이메일 수신은 유출 증거가 아니다
- 회사 정보가 외부 채널로 "발신"된 경우에만 의심한다
- 은폐 시도(삭제, 암호화, 확장자 변경)가 동반되면 가중한다
- Counter-evidence Agent의 반증이 있으면 감점한다

교차 대조 로직 (코드)

```
def cross_reference(suspicious_channels: list, sensitive_files: list) -> list:
    """
    유출 경로로 나간 파일 중 민감 파일이 포함된 경우를 매핑한다.

    suspicious_channels: [{"email_id": ..., "attachments": [...], "channel_type":
    "protonmail"}]
    sensitive_files:      [{"file_id": ..., "filename": ..., "sensitivity_score":
    0.9}]

    반환: [{"email_id": ..., "sensitive_file": ..., "match_reason": ...}]
    """
    results = []
    sensitive_filenames = {f["filename"].lower() for f in sensitive_files}

    for channel in suspicious_channels:
        for attachment in channel.get("attachments", []):
            if attachment["filename"].lower() in sensitive_filenames:
                results.append({
                    "email_id": channel["email_id"],
                    "channel_type": channel["channel_type"],
                    "sensitive_file": attachment["filename"],
                    "sent_at": channel["sent_at"],
                    "match_reason": f"민감 파일 '{attachment['filename']}' 이
{channel['channel_type']}로 발송됨"
                })
    return results
```

6. STEP 1 — Baseline Agent

역할

퇴사일 기준 90일 이전의 데이터로 "정상 행동 패턴"을 수립한다. 이 기준선이 없으면 이후 모든 에이전트가 무엇이 "이상"인지 판단할 수 없다.

내부 동작 구조 (Deep Agent 패턴)

[계획]

Main으로부터 task 수신 후 LLM이 분석 순서 결정:

- 1) 외부 이메일 발신 이력 조회

- 2) 파일 실행 이력 조회
- 3) USB/외부 저장장치 이벤트 조회
- 4) 업무 시간대 분포 확인
- 5) 데이터 충분성 검증 (has_sufficient_data)

[실행]

결정한 순서대로 Tool 호출

→ get_email_history() → get_file_access_history() → get_activity_events()

[반환]

baseline_profile dict 구조화

→ Main Agent에게 반환

→ Main Agent가 State에 저장 후 STEP 2/3/4에 task 전달

Input

```
{
  "subject_name": "이지수",
  "resignation_date": "2021-02-28",
  "analysis_start": "2020-11-30",    # resignation_date - 90일
  "source_label": "HYENA CTF"
}
```

Output

```
{
  "normal_external_email_count": 3,      # 평소 하루 평균 외부 발신 수
  "normal_external_domains": [          # 평소 연락하던 외부 도메인
    "gmail.com", "naver.com", "hytint.com"
  ],
  "normal_file_access_count": 12,        # 하루 평균 파일 실행 수
  "normal_file_types": [".hwp", ".xlsx", ".pdf"], # 주로 다루던 파일 유형
  "usb_ever_used": False,                # USB 사용 이력 있는가
  "normal_working_hours": "09:00-18:00", # 평소 업무 시간대
  "has_sufficient_data": True            # 기준선 수립에 충분한 데이터가 있는가
}
```

판단 기준

항목	계산 방법	주의
외부 이메일 평균	analysis_start ~ resignation_date-30일 기간 발신 수 / 일수	마지막 30일은 제외 (이상 기간)
파일 접근 평균	file_access_logs에서 동일 기간 run_count 합산 / 일수	
USB 사용 여부	activity_events에서 USB 관련 event_type 존재 여부	

항목	계산 방법	주의
업무 시간대	email_messages의 sent_at 시간대 분포	

사용하는 Tool 함수

```

get_email_history(user_name, date_from, date_to)
→ email_messages에서 해당 기간 발신 이력 조회

get_file_access_history(user_name, date_from, date_to)
→ file_access_logs에서 해당 기간 파일 실행 이력 조회

get_activity_events(user_name, date_from, date_to, event_types=None)
→ activity_events에서 이벤트 이력 조회

```

DB 쿼리 예시

```

-- 외부 발신 이메일 기준선
SELECT DATE(sent_at) as day, COUNT(*) as cnt
FROM email_messages
WHERE sender ILIKE '%이지수%'
      AND sent_at BETWEEN '2020-11-30' AND '2021-01-29'
      AND recipients_to::text NOT ILIKE '%hb.%' -- 내부 도메인 제외
GROUP BY DATE(sent_at)
ORDER BY day;

```

System Prompt

분석 대상: {subject_name}
 분석 기간: {analysis_start} ~ {resignation_date - 30일} (기준선 수립 기간)

다음 Tool들을 사용해서 이 기간의 "정상 행동 패턴"을 파악하세요.
 정상 패턴이란 퇴사 의심 행동이 없었던 평소의 행동을 의미합니다.

시작 전에 수행할 분석 항목을 순서대로 먼저 나열하고, 완료할 때마다 체크하면서 진행하세요.

수집할 정보:

1. 외부 이메일 발신 빈도와 주요 수신자 도메인
2. 파일 실행 기록의 일평균 수량과 주요 파일 유형
3. USB 또는 외부 저장장치 사용 이력
4. 주요 업무 시간대

주의: 분석 기간의 "마지막 30일"은 이미 의심 기간일 수 있으므로 기준선에서 제외하세요.
 데이터가 부족하면 has_sufficient_data를 false로 반환하고 이유를 명시하세요.

7. STEP 2 — 유출 행위 분석 Agent

역할

퇴사 전 90일 이내에 **회사 정보가 외부로 나갈 수 있는 채널**을 탐지한다. 이메일, 메신저, 익명 채널을 모두 확인한다.

내부 동작 구조 (Deep Agent 패턴)

[계획]

Main으로부터 task 수신 후 LLM이 탐지 순서 결정:

- 1) 익명 채널(ProtonMail/tmpbox) 이메일 조회 - 위험도 높아 우선 확인
- 2) 기준선 대비 외부 발신량 비교
- 3) 업무 외 시간대 외부 발신 확인
- 4) 첨부파일 포함 외부 발신 확인
- 5) 메신저 민감 키워드 탐지

[실행]

결정한 순서대로 Tool 호출

→ get_anonymous_channel_emails() → get_external_emails()
→ get_email_attachments() → get_messenger_logs()

[반환]

suspicious_channels 리스트 구조화 (channel_type, risk_weight 포함)
→ State에 저장 → Main Agent로 반환

Input

```
{
  "subject_name": "이지수",
  "baseline_profile": {...},          # STEP 1 결과 (정상 기준선)
  "analysis_start": "2020-11-30",
  "resignation_date": "2021-02-28"
}
```

Output

```
[
  {
    "channel_type": "protonmail",      # 채널 유형
    "email_id": "uuid-...",           # email_messages.id
    "sender": "hb.jisu.lee@gmail.com", # 발신자 (본인 Gmail)
    "recipient": "pyungsea@protonmail.com",
    "subject": "이지수 과장님께",
    "sent_at": "2021-01-22T07:40:00Z",
    "has_attachment": False,
    "attachments": [],
    "body_preview": "RE: 이지수 과장님께...",
  }
]
```

```

        "highlight_keywords": ["protonmail", "과장님"],
        "suspicion_reason": "ProtonMail 익명 채널로 수신된 이메일에 회신",
        "risk_weight": 30
    },
    {
        "channel_type": "messenger_leak",
        "messenger_log_ids": ["uuid-..."],
        "participants": ["구매팀 장국주 팀장", "구매팀 이지수 과장"],
        "topic_summary": "업체 단가 정보 공유",
        "suspicion_reason": "메신저로 단가 정보를 팀장에게 전달",
        "risk_weight": 20
    }
]

```

탐지 대상 채널 유형

channel_type	정의	탐지 방법
protonmail	ProtonMail 주소로 발신/수신	sender/recipient ILIKE '%protonmail%'
tmpbox	일회용 임시 메일 서비스	tmpbox.net, moakt.com 등
personal_gmail	개인 Gmail 계정으로 발신	본인 hb.*.gmail.com이 발신자인 경우
anonymous_channel	익명 채널 전반	ProtonMail, Tutanota, Guerrilla Mail 등
messenger_leak	메신저로 민감 정보 공유	messenger_logs에서 키워드 탐지
baseline_anomaly	기준선 대비 외부 발신 급증	평균 대비 3배 이상 증가 시

판단 기준 — 기준선 대비 이상 여부

```

def is_anomalous_email_activity(daily_count, baseline_avg):
    """
    기준선 대비 3배 이상이면 이상으로 판단
    단, 절대 수가 3건 미만이면 노이즈로 처리
    """
    if daily_count < 3:
        return False
    return daily_count > baseline_avg * 3

```

사용하는 Tool 함수

```

get_external_emails(user_name, date_from, date_to)
    → 외부 수신자로 발송된 이메일 목록

get_anonymous_channel_emails(date_from, date_to)
    → protonmail, tmpbox 등 익명 채널 이메일

get_messenger_logs(user_name, date_from, date_to, keywords=None)

```

→ 메신저 대화 기록 (keywords로 필터링 가능)

```
get_email_attachments(email_id)
```

→ 특정 이메일의 첨부파일 목록

DB 쿼리 예시

```
-- 익명 채널 이메일 탐지
SELECT id, subject, sender, recipients_to, sent_at, has_attachments
FROM email_messages
WHERE sent_at BETWEEN '2020-11-30' AND '2021-02-28'
  AND (
    sender ILIKE '%protonmail%'
    OR sender ILIKE '%tmpbox%'
    OR sender ILIKE '%moakt%'
    OR sender ILIKE '%guerrillamail%'
    OR recipients_to::text ILIKE '%protonmail%'
  )
ORDER BY sent_at;

-- 메신저에서 민감 키워드 탐지
SELECT chat_title, sender, message, sent_at
FROM messenger_logs
WHERE message ILIKE '%단가%'
  OR message ILIKE '%계약%'
  OR message ILIKE '%견적%'
  OR message ILIKE '%비밀%'
ORDER BY sent_at;
```

System Prompt

분석 대상: {subject_name}
 기준선: {baseline_profile}
 분석 기간: {analysis_start} ~ {resignation_date}

다음 Tool들을 사용해서 이 기간에 회사 정보가 외부로 유출됐을 가능성이 있는 모든 채널을 탐지하세요.

시작 전에 탐지할 채널 유형을 우선순위 순서로 나열하고, 완료할 때마다 체크하면서 진행하세요.

탐지 우선순위:

1. ProtonMail, tmpbox 등 익명/추적 불가 채널 사용
2. 기준선 대비 외부 발신 급증 (3배 이상)
3. 업무 외 시간대(22:00 이후, 주말)에 외부 발신
4. 첨부파일이 포함된 외부 발신
5. 메신저에서 단가, 계약, 거래처 정보 언급

주의:

- 뉴스레터 수신(newsletters, noreply 등)은 유출과 무관하므로 제외하세요
- 정상 거래처와의 업무 메일은 기준선과 비교해서 판단하세요
- 각 의심 항목에 suspicion_reason과 risk_weight(0-40)를 반드시 포함하세요

8. STEP 3 — 민감 파일 분류 Agent

역할

DB에 적재된 모든 문서 중에서 **기밀성이 높은 파일을 식별**한다. 벡터 유사도 검색으로 내용 기반 분류, Neo4j로 핵심 엔티티 포함 파일을 추출한다.

내부 동작 구조 (Deep Agent 패턴)

[계획]

Main으로부터 task 수신 후 LLM이 분류 순서 결정:

- 1) Qdrant 벡터 검색 - 민감 카테고리 5종 쿼리
- 2) Neo4j 그래프 검색 - 핵심 엔티티 언급 파일 추출
- 3) 두 결과 병합 및 sensitivity_score 계산
- 4) 0.7 미만 항목 제거

[실행]

- search_vector_db() × 5회 (카테고리별)
- get_files_by_entity()
- get_file_metadata() (상위 결과 메타데이터 보완)

[반환]

- sensitive_files 리스트 구조화 (sensitivity_score, category 포함)
- State에 저장 → Main Agent로 반환

Input

```
{
  "subject_name": "이지수",
  "source_label": "HYENA CTF"
}
```

Output

```
[
  {
    "file_id": "uuid-...",
    "filename": "2021년 구매 단가표.xlsx",
    "relative_path": "HYENA CTF/구매팀_이지수/C/Users/HB/Desktop/...",
    "extension": ".xlsx",
    "sensitivity_score": 0.92,      # 0.0 ~ 1.0
  }
]
```

```

    "sensitivity_category": "단가/계약",
    "matched_keywords": ["단가", "계약", "원가"],
    "related_entities": ["HYT인터내셔널", "가나트리"],
    "highlight_keywords": ["단가표", "계약", "원가"]
  }
]

```

민감도 분류 기준

sensitivity_category	대표 키워드	sensitivity_score 범위
계약/단가	단가표, 계약서, 원가, 견적서, 거래처	0.85 ~ 1.0
인사/내부	인사, 급여, 평가, 발령, 조직도	0.80 ~ 1.0
영업/전략	영업전략, 매출, 수주, 경쟁사	0.75 ~ 0.95
기술/설계	설계도, 특허, 소스코드, 기술서	0.80 ~ 1.0
재무	예산, 손익, 매출, 원가율	0.75 ~ 0.95
일반 업무	회의록, 기안서, 보고서	0.30 ~ 0.60

사용하는 Tool 함수

```

search_vector_db(query_text, top_k=50, threshold=0.75)
  → Qdrant에서 의미적으로 유사한 청크 검색
  → query_text 예: "기밀 단가 계약 거래처 원가"

get_files_by_entity(entity_names, source_label)
  → Neo4j에서 특정 엔티티를 언급하는 파일 목록 조회

get_file_metadata(file_id)
  → files 테이블에서 파일 메타데이터 조회

```

Qdrant 쿼리 예시

```

# 민감 카테고리별 검색 쿼리
SENSITIVE_QUERIES = [
    "계약서 단가표 견적서 거래처 원가",
    "인사 급여 평가 조직도 발령",
    "영업전략 매출 수주 경쟁사",
    "설계 특허 소스코드 기술",
    "재무 예산 손익 매출",
]

for query in SENSITIVE_QUERIES:
    results = qdrant_client.search(
        collection_name="hyena_content_chunks",
        query_vector=embed(query),
    )

```

```

        limit=20,
        score_threshold=0.75
    )

```

Neo4j 쿼리 예시

```

-- 핵심 엔티티(조직, 인물)를 언급하는 파일 찾기
MATCH (f:GNode {node_type:'file'})-[:MENTIONS]->(e:GNode)
WHERE e.node_type IN ['organization', 'person']
      AND e.label IN ['HYT인터내셔널', '가나트리', '행복의류']
RETURN f.label as filename, f.node_id as file_id,
       collect(e.label) as mentioned_entities
ORDER BY size(collect(e.label)) DESC
LIMIT 30

```

System Prompt

당신은 디지털 포렌식 전문가입니다.

분석 대상 {subject_name}의 PC에 있는 모든 파일 중에서
기밀성이 높은 파일을 식별해야 합니다.

시작 전에 민감 파일 분류 절차를 순서대로 나열하고, 완료할 때마다 체크하면서 진행하세요.

다음 Tool들을 순서대로 사용하세요:

1. search_vector_db() - 민감 내용(단가, 계약, 인사 등) 키워드로 벡터 검색
2. get_files_by_entity() - Neo4j에서 핵심 거래처/인물을 언급하는 파일 검색
3. 두 결과를 합쳐서 sensitivity_score 계산

sensitivity_score 계산 방법:

- 벡터 유사도 점수 (0.75 이상만 포함)
- 민감 카테고리 키워드 직접 포함 여부 (+0.1)
- 핵심 엔티티 언급 횟수 (언급당 +0.05, 최대 +0.2)

최종적으로 sensitivity_score 0.7 이상인 파일만 반환하세요.

각 파일에 sensitivity_category와 highlight_keywords를 반드시 포함하세요.

9. STEP 4 — 행동 패턴 분석 Agent

역할

Baseline과 비교해서 **퇴사 전 행동이 얼마나 비정상적이었는지** 분석한다. WHO(어떤 비정상 행동을 했나) + WHEN(언제 집중됐나)를 함께 분석한다.

내부 동작 구조 (Deep Agent 패턴)

[계획]

Main으로부터 task 수신 후 LLM이 분석 순서 결정:

- 1) 파일 접근 로그 조회 – 기준선과 비교
- 2) 삭제 파일 목록 조회 – 은폐 시도 탐지
- 3) 활동 이벤트 타임라인 조회
- 4) WHO 분석: 비정상 접근 / 삭제 / 대용량 / 업무 외 시간대
- 5) WHEN 분석: 날짜별 anomaly_score 산출 및 highlight_dates 결정

[실행]

→ get_file_access_logs() → get_deleted_files()
→ get_activity_events_timeline()

[반환]

behavior_anomalies dict 구조화 (who_analysis, when_analysis 포함)
→ State에 저장 → Main Agent로 반환

Input

```
{
  "subject_name": "이지수",
  "baseline_profile": {...},      # STEP 1 결과
  "analysis_start": "2020-11-30",
  "resignation_date": "2021-02-28"
}
```

Output

```
{
  "who_analysis": {
    "accessed_unusual_files": [      # 평소 안 보던 파일/폴더 접근
      {
        "filename": "2021_전체단가표.xlsx",
        "last_run_at": "2021-01-22T14:30:00Z",
        "reason": "구매팀 이지수 업무 범위 외 파일 접근"
      }
    ],
    "deleted_files": [              # 삭제된 파일
      {
        "original_filename": "업무관련 서식.zip",
        "deleted_at": "2021-01-22T04:08:55Z",
        "file_size_bytes": 34583034,
        "reason": "퇴사 전 34MB 압축파일 삭제 – 은폐 의심"
      }
    ],
    "out_of_hours_activity": []     # 업무 외 시간 활동
  },
  "when_analysis": {
    "peak_suspicious_date": "2021-01-22",  # 가장 의심 행동이 집중된 날짜
  }
}
```



```

        "timeline": [                                     # 날짜별 이상 활동 요약
            {
                "date": "2021-01-22",
                "events": ["외부 메일 3건", "파일 삭제 4건", "ProtonMail 수신"],
                "anomaly_score": 0.9
            }
        ],
    },
    "highlight_dates": ["2021-01-22", "2021-01-25"]
}

```

판단 기준

WHO 분석 기준:

항목	이상 판정 조건
비정상 파일 접근	기준선에 없던 파일 유형 접근
파일 삭제	deleted_files에 기록된 항목 전체
대용량 파일 처리	10MB 이상 파일을 접근/삭제한 경우
업무 외 시간 활동	22:00 이후 또는 주말 파일 접근

WHEN 분석 기준:

```

def calculate_daily_anomaly_score(date, events, baseline):
    score = 0.0

    # 외부 발신이 기준선 3배 이상
    if events["external_emails"] > baseline["normal_external_email_count"] * 3:
        score += 0.3

    # 파일 삭제 발생
    if events["deleted_files"] > 0:
        score += 0.4

    # 익명 채널 사용
    if events["anonymous_channel"]:
        score += 0.5

    # 업무 외 시간대 활동
    if events["out_of_hours"]:
        score += 0.2

    return min(score, 1.0)

```

사용하는 Tool 함수

```

get_file_access_logs(user_name, date_from, date_to)
→ file_access_logs에서 파일 실행 이력 조회

get_deleted_files(user_name, date_from, date_to)
→ deleted_files에서 삭제 파일 조회

get_activity_events_timeline(user_name, date_from, date_to)
→ activity_events를 날짜별로 그룹화해서 반환

```

System Prompt

분석 대상: {subject_name}
 기준선: {baseline_profile}
 분석 기간: {analysis_start} ~ {resignation_date}

시작 전에 WHO/WHEN 분석 항목을 순서대로 나열하고, 완료할 때마다 체크하면서 진행하세요.

다음 두 가지 관점에서 이상 행동을 분석하세요.

[WHO 분석 – 어떤 비정상 행동을 했나]

- 평소와 다른 파일/폴더 접근 패턴
- 삭제된 파일 (deleted_files 전체 포함)
- 대용량 파일(10MB 이상) 처리
- 업무 외 시간대(22:00 이후, 주말) 활동

[WHEN 분석 – 언제 집중됐나]

- 날짜별로 의심 이벤트를 집계해서 anomaly_score(0-1) 산출
- anomaly_score가 높은 날짜를 highlight_dates로 반환
- 타임라인에 각 날짜의 이상 이벤트 목록 포함

주의:

- 파일 삭제는 그 자체로 은폐 시도의 증거입니다. 반드시 포함하세요.
- 기준선이 없는 데이터는 "데이터 부족"으로 명시하고 계속 진행하세요.

10. STEP 5 — Counter-evidence Agent

역할

STEP 2~4에서 나온 의심 항목들에 대해 **반증을 찾고 오답을 제거**한다. 의심 항목이 정상 업무 행위로 설명될 수 있는지 검증한다.

내부 동작 구조 (Deep Agent 패턴)

[계획]
 Main으로부터 cross_reference 결과 + 전체 의심 항목 수신
 LLM이 검증 순서 결정 (위험도 높은 항목 우선):

- 1) cross_reference hit 항목 – 기밀 파일 x 익명 채널 매칭
- 2) 익명 채널 사용 항목 – 수신만 했는지 발신도 했는지 확인
- 3) 파일 삭제 항목 – 임시/캐시 파일 여부 확인
- 4) 외부 이메일 증가 항목 – 계절적/업무적 맥락 확인

[실행]

의심 항목별로 Tool 호출하여 반증 탐색:

→ check_email_history_with_sender() → get_email_context()
 → check_seasonal_pattern()

[반환]

verified_findings 리스트 (verified=True/False, confidence 포함)

→ State에 저장 → Main Agent로 반환
 → Main Agent가 리스크 스코어링으로 이동

Input

```
{
  "subject_name": "이지수",
  "suspicious_channels": [...],      # STEP 2 결과
  "sensitive_files": [...],          # STEP 3 결과
  "behavior_anomalies": {...},       # STEP 4 결과
  "cross_reference": [...]           # Main Agent 교차 대조 결과
}
```

Output

```
[
  {
    "finding_id": "finding_001",
    "original_finding": {
      "type": "protonmail",
      "description": "ProtonMail로 회신"
    },
    "counter_evidence": None,          # 반증 없음
    "verified": True,                 # 유효한 의심 항목
    "confidence": 0.85,
    "final_reason": "ProtonMail 암호화 채널 사용 – 추적 회피 의도 의심",
    "highlight_keywords": ["ProtonMail", "과장님"],
    "email_id": "uuid-..."          # 프론트에서 원본 보기용
  },
  {
    "finding_id": "finding_002",
    "original_finding": {
      "type": "external_email",
      "description": "외부 발신 증가"
    },
    "counter_evidence": "분석 기간이 신규 시즌 발주 시기와 일치. 거래처 연락 증가는 정상 업무 범주.",
  }
]
```

```
        "verified": False,                # 오탐 - 제거
        "confidence": 0.2,
        "final_reason": "정상 업무 패턴으로 판단"
    }
]
```

반증 탐색 기준

의심 유형	반증 조건 (verified=False 처리)
외부 이메일 증가	동일 거래처와 이전에도 정기적 연락 이력 있음
파일 접근 증가	시즌 발주, 결산 시기 등 업무 맥락 존재
익명 채널 수신	먼저 발신한 적 없고, 수신만 한 경우
파일 삭제	임시파일, 캐시, 설치파일 등 비업무용
업무 외 시간 활동	야근 이력이 기준선에서도 존재

사용하는 Tool 함수

```
check_email_history_with_sender(sender, user_name, date_from, date_to)
    → 특정 발신자와의 이전 교신 이력 확인

get_email_context(email_id)
    → 특정 이메일의 스레드 전체 조회

check_seasonal_pattern(user_name, month)
    → 특정 월에 이전 해에도 유사 패턴 있었는지 확인
```

System Prompt

당신은 검사 역할의 포렌식 전문가입니다.
다음 의심 항목들에 대해 반증을 찾아야 합니다.

시작 전에 검증할 의심 항목 목록을 우선순위 순서로 나열하고, 완료할 때마다 체크하면서 진행 하세요.

의심 항목 목록:
{suspicious_items}

각 항목에 대해:

- 정상 업무로 설명 가능한가? → Tool을 사용해서 확인
- 기준선 데이터와 비교했을 때 실제로 이상한가?
- 맥락상 업무 필요성이 있는가?

반증을 찾으면 verified=False, counter_evidence에 근거 명시
반증이 없으면 verified=True, confidence를 0.7~1.0 범위로 산출

주의:

- ProtonMail, tmpbox 등 익명 채널 사용은 반증이 거의 없습니다.
수신만 한 경우에만 `verified=False` 가능합니다.
- 파일 삭제는 임시/캐시 파일이 아닌 경우 `verified=True`를 유지하세요.
- 확신이 없으면 `verified=True`로 유지하고 `confidence`를 낮게 설정하세요.

11. 리스크 스코어링 (Main Agent 내부)

역할

`verified=True`인 모든 항목을 종합해서 최종 리스크 점수를 계산한다. 점수에 따라 HIGH / MEDIUM / LOW / CLEAN 을 판정한다.

가중치 테이블

항목	가중치	조건
기밀 파일이 익명 채널로 발신됨 (교차 대조 hit)	+40	<code>cross_reference</code> 에 매칭 존재
은폐 시도 탐지 (삭제, 확장자 변경 등)	+30	<code>deleted_files</code> 또는 은폐 이벤트 존재
Baseline 대비 이상 행동	+20	<code>anomaly_score</code> 0.7 이상인 날짜 존재
익명 채널 사용만 (파일 매칭 없음)	+15	<code>channel_type</code> in ['protonmail', 'tmpbox']
Counter-evidence 반증	-20	<code>verified=False</code> 항목당

판정 기준

```
def calculate_verdict(risk_score: int) -> str:
    if risk_score >= 81:
        return "HIGH" # 유출 확정에 가까움
    elif risk_score >= 61:
        return "MEDIUM" # 강한 유출 의심
    elif risk_score >= 41:
        return "LOW" # 조사 필요
    else:
        return "CLEAN" # 클린 인증서 발급
```

스코어 계산 예시

이지수 분석 결과:

- cross_reference hit (단가표 → ProtonMail) +40
- deleted_files 4건 존재 +30
- anomaly_score 0.9인 날짜 존재 +20
- Counter-evidence 없음 0

최종 점수: 90점 → HIGH

12. 최종 출력 포맷

유출 있음 (HIGH/MEDIUM/LOW)

```
{
  "report_type": "EXFILTRATION_SUSPECTED",
  "verdict": "HIGH",
  "risk_score": 90,
  "subject": {
    "name": "이지수",
    "position": "과장",
    "hire_date": "2019-03-01",
    "resignation_date": "2021-02-28"
  },
  "summary": "퇴사 전 30일 이내에 기밀 파일을 ProtonMail 익명 채널로 발신하고, 관련 파일 4건을 삭제한 정황이 확인됨",
  "suspicious_emails": [ # 프론트에서 원본 이메일 뷰어용
    {
      "email_id": "uuid-...",
      "subject": "이지수 과장님께",
      "sender": "pyungsea@protonmail.com",
      "sent_at": "2021-01-22",
      "reason": "ProtonMail 익명 채널",
      "highlight_keywords": ["ProtonMail", "과장님께"]
    }
  ],
  "suspicious_files": [ # 파일 하이라이트용
    {
      "file_id": "uuid-...",
      "filename": "2021년 구매 단가표.xlsx",
      "sensitivity_score": 0.92,
      "reason": "기밀 단가 정보 포함",
      "highlight_keywords": ["단가", "계약"]
    }
  ],
  "deleted_files": [ # 삭제된 파일 목록
    {
      "original_filename": "업무관련 서식.zip",
      "deleted_at": "2021-01-22",
      "file_size_bytes": 34583034,
      "reason": "퇴사 전 대용량 파일 삭제"
    }
  ]
}
```

```

    }
  ],
  "timeline": [                                # 타임라인 시각화용
    {
      "date": "2021-01-22",
      "events": ["외부 메일 3건", "파일 삭제 4건"],
      "anomaly_score": 0.9
    }
  ],
  "evidence_network": {                        # Neo4j 시각화용
    "nodes": [...],
    "edges": [...]
  }
}

```

유출 없음 (CLEAN)

```

{
  "report_type": "CLEAN_CERTIFICATE",
  "verdict": "CLEAN",
  "risk_score": 15,
  "subject": {...},
  "summary": "분석 기간 내 데이터 유출 의심 행위가 발견되지 않음",
  "analysis_summary": {
    "emails_analyzed": 120,
    "files_analyzed": 300,
    "anomalies_found": 0,
    "false_positives_removed": 2
  },
  "issued_at": "2026-05-14T10:00:00Z"
}

```

13. 구현 기술 스택

에이전트 프레임워크

LangGraph (권장) 또는 Claude Agent SDK

- └─ 각 에이전트 = LangGraph 노드
- └─ State = InvestigationState TypedDict
- └─ 병렬 실행 = LangGraph의 fan-out/fan-in 패턴

LLM 모델

Claude claude-sonnet-4-6 (권장)

- └─ 긴 컨텍스트 처리 능력

- └─ Tool use (function calling) 지원
- └─ 한국어 처리 우수

캡스톤 한계: 로컬 LLM(Ollama) 사용 시 성능 저하 가능

Tool 구현 위치

```
agent/
├── tools/
│   ├── rdb_tools.py      # PostgreSQL 조회 함수
│   ├── vector_tools.py   # Qdrant 벡터 검색 함수
│   └── graph_tools.py    # Neo4j Cypher 쿼리 함수
├── nodes/
│   ├── baseline.py       # STEP 1
│   ├── exfiltration.py   # STEP 2
│   ├── sensitive_files.py # STEP 3
│   ├── behavior.py       # STEP 4
│   └── counter_evidence.py # STEP 5
└── graph.py              # LangGraph 메인 그래프
```

DB 직접 연결 (HTTP API 없이)

```
# 에이전트는 HTTP 호출 없이 DB 클라이언트를 직접 사용
import psycopg2
from qdrant_client import QdrantClient
from neo4j import GraphDatabase

# Tool 함수 예시
def query_external_emails(user_name: str, date_from: str, date_to: str) -> list:
    conn = get_pg_conn()
    cur = conn.cursor()
    cur.execute("""
        SELECT id, subject, sender, recipients_to, sent_at, has_attachments
        FROM email_messages
        WHERE sender ILIKE %s
              AND sent_at BETWEEN %s AND %s
    """, [f"%{user_name}%", date_from, date_to])
    return cur.fetchall()
```

14. Tool 함수 명세

PostgreSQL Tool 함수

함수명	파라미터	반환	사용 에이전트
get_email_history	user_name, date_from, date_to	이메일 목록	STEP 1, 2
get_external_emails	user_name, date_from, date_to	외부 발신 이메일	STEP 2
get_anonymous_channel_emails	date_from, date_to	ProtonMail/tmpbox 이메일	STEP 2
get_email_attachments	email_id	첨부파일 목록	STEP 2
get_messenger_logs	user_name, date_from, date_to, keywords	메신저 로그	STEP 2
get_file_access_logs	user_name, date_from, date_to	파일 실행 이력	STEP 1, 4
get_deleted_files	user_name, date_from, date_to	삭제 파일	STEP 4
get_activity_events_timeline	user_name, date_from, date_to	이벤트 타임라인	STEP 1, 4
get_email_thread	email_id	이메일 스레드 전체	STEP 5

Qdrant Tool 함수

함수명	파라미터	반환	사용 에이전트
search_vector_db	query_text, top_k, threshold	유사 청크 목록	STEP 3
get_chunk_by_file	file_id	특정 파일의 모든 청크	STEP 3

Neo4j Tool 함수

함수명	파라미터	반환	사용 에이전트
get_files_by_entity	entity_names, source_label	엔티티 언급 파일 목록	STEP 3
query_graph	cypher_query	그래프 쿼리 결과	STEP 5
get_related_nodes	node_id, rel_types, depth	연관 노드 목록	STEP 5

작성일: 2026-05-14 버전: v1.0